Field Hours

# CSCE 676 Data Mining and Analysis

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

## Look at the table first

You are handed a table, and the first computation is a description, not a model. Count the rows, the columns, and the empty cells. If 15 of 100 rows leave a column empty, the missingness of that column is 0.15, and a mean computed after dropping them is a mean of 85 rows. You record 15 and 85. An algorithm that drops empty rows is no longer describing all 100.

Columns also lie about scale. A measurement in the thousands and a measurement between 0 and 1 will not contribute fairly to a sum of squares. You divide each column by its own spread, or you subtract the column mean and divide by the column standard deviation, before you cluster. If one column has standard deviation 50 and another has standard deviation 2, the first column will own an unscaled Euclidean distance.

Describe the table before a model.

1. Count rows, columns, and holes

2. Record missingness of 0.15

3. Standardize columns with huge spread

4. Only then cluster or classify

### A column missing in 15 rows

1. The table has 100 rows. One column is empty in 15 of them.

2. Missingness is 15/100 = 0.15. A mean that drops those rows uses 85 rows, not 100.

3. You write 15, 85, and 0.15 next to the column. A later step that refuses empty cells is describing a different population.

Problem 1. 15 of 100 rows leave a column empty, and you drop them before the mean. What are the missingness and the number of rows in that mean?

Answer. Missingness is 0.15. The mean uses 85 rows. You record both, because the algorithm no longer describes all 100 rows.

Problem 2. One column has standard deviation 50 and another has standard deviation 2. What happens to an unscaled Euclidean distance, and what do you do first?

Answer. The column with spread 50 dominates the sum of squares. You standardize, subtracting each column's mean and dividing by its own standard deviation, before you cluster.

Lecture 2 · 90 minutes

# Frequent itemsets and Apriori

A frequent itemset is a set of items whose support is at least a threshold you choose. Support is the fraction of transactions that contain the set. Equivalently you can count: with 4 transactions, support 0.5 means the set occurs in at least 2 transactions. Apriori's pruning rule is the mechanism that makes the search smaller. If a set occurs in only 1 transaction, every superset of it occurs in at most 1, so you never generate those supersets.

You start with single items, drop those below the count, and join the survivors into pairs. Take four baskets: milk bread eggs; milk bread; bread eggs; milk eggs. With minimum count 2, milk, bread, and eggs each occur 3 times and all survive. The pairs milk-bread, milk-eggs, and bread-eggs each occur 2 times and survive. The triple milk-bread-eggs occurs in 1 basket and dies.

Grow sets only from frequent sets.

Count single items -> Drop counts below 2 -> Join survivors into pairs -> Stop when the triple dies

## Four baskets, minimum count 2

1. Baskets are {milk, bread, eggs}, {milk, bread}, {bread, eggs}, and {milk, eggs}. The minimum count is 2.

2. milk, bread, and eggs each occur 3 times. Every pair among them occurs 2 times, so all three pairs survive.

3. The triple occurs in 1 basket. Apriori does not extend it, because every superset of an infrequent set is infrequent.

Problem 1. A set occurs in 1 of 4 baskets, and the minimum count is 2. Why do you skip its supersets?

Answer. A superset cannot occur more often than the set itself, so every superset occurs in at most 1 basket. That is below 2, and Apriori never generates those supersets.

Problem 2. Using the four baskets and minimum count 2, which pairs survive, and what is the count of {milk, bread, eggs}?

Answer. milk-bread, milk-eggs, and bread-eggs each occur 2 times and survive. The triple occurs once, in the first basket only, so it is not frequent.

Lecture 3 · 90 minutes

# Confidence and lift

You split an itemset into a left side and a right side to make a rule. Confidence of X implies Y is support of X union Y divided by support of X. It answers how often Y shows up in the transactions that already have X. From the four baskets, milk and bread occur together in 2, and milk occurs in 3, so confidence of milk implies bread is 2/3.

Lift compares that confidence with how common Y is anyway. Lift is confidence of X implies Y divided by support of Y. Bread occurs in 3 of 4 baskets, support 3/4. Lift of milk implies bread is (2/3) divided by (3/4), which is 8/9, a bit under 1. A lift near 1 means the rule restates the base rate. You would rather see a lift well

above 1 before you treat the rule as a dependence.

Frequency is not dependence.

Confidence

Divide by support of X

Milk implies bread is 2/3

Ignores how common Y is

Lift

Divide confidence by support of Y

Bread already has support 3/4

Lift is 8/9

### Milk implies bread

1. The pair milk and bread occurs in 2 baskets. Milk occurs in 3. Confidence is 2/3.

2. Bread occurs in 3 of 4 baskets, so its support is 3/4.

3. Lift is (2/3) / (3/4) = 8/9, which is under 1. The pair is frequent at count 2 and still not a positive dependence.

Problem 1. In the four baskets, what is the confidence of milk implies bread, and what is the support of that pair?

Answer. Confidence is 2/3, because the pair occurs in 2 baskets and milk occurs in 3. Support of the pair is 2/4 = 1/2. Confidence divides by the left side, not by all 4 baskets.

Problem 2. Bread's support is 3/4 and the confidence of milk implies bread is 2/3. What is the lift, and does a minimum count of 2 make this a dependence?

Answer. Lift is (2/3) / (3/4) = 8/9, which is under 1. The pair meets count 2, but lift near 1 means the rule restates the base rate. Frequency did not give you a dependence.

Lecture 4 · 90 minutes

## K-means on a line

K-means alternates two steps you can implement directly. Assign each row to the nearest centroid, then replace each centroid by the mean of its rows. Stop when the assignment does not change. Points 1, 2, 10, and 11, with k = 2 and initial centroids 1 and 10, form two pairs. The updated centroids are 1.5 and 10.5. The next assignment keeps those pairs, so this run stops after that update.

The result depends on the initial centroids and on k, which you chose. Two different starts can produce two different partitions of the same 4 points, and the algorithm will report both as converged. You keep the partition with the smaller within-cluster sum of squares. Empty clusters are a real case: if a centroid wins 0 rows, you cannot take its mean. Place it again, or reduce k.

Assign, average, and stop.

1. Start centroids at 1 and 10

2. Assign each point to the nearer

3. Replace centroids by means

4. Stop when pairs do not move

### Update 1, 2, 10, and 11

1. Initial centroids are 1 and 10. Points 1 and 2 are nearer to 1. Points 10 and 11 are nearer to 10.

2. The new means are (1+2)/2 = 1.5 and (10+11)/2 = 10.5.

3. Reassignment leaves the same pairs, so the run has converged. You do not average again unless the membership changes.

Problem 1. Points are 1, 2, 10, and 11. Initial centroids are 1 and 10. What are the two means after one update?

Answer. 1.5 and 10.5. The pairs are {1, 2} and {10, 11}, and each mean is the average of its pair. The next assignment does not change, so the run stops.

Problem 2. A centroid is assigned 0 of the 4 points. Why can you not leave it where it is and still call the update a mean?

Answer. A mean of no rows is undefined. You place that centroid again or you reduce k. Leaving the old coordinate pretends it still represents points.

Lecture 5 · 90 minutes

## Squared error and the choice of k

You justify a choice of k by the within-cluster sum of squared errors. For centroids 1.5 and 10.5 the four squared errors are each 0.25, so the sum is 1. One cluster around the mean 6 has sum (5 squared) + (4 squared) + (4 squared) + (5 squared) = 82. Lower is better only when k is fixed. If you set k to 4, each point is a centroid and the sum is 0, which is perfect and useless.

Squared Euclidean distance is the mechanism inside that sum, so every coordinate must be on a scale you can square. Standardize 2 columns that use different units before you trust a comparison. Manhattan distance sums absolute deviations and does not square a wild coordinate. If you change the distance, the mean may no longer be the right center. K-means as stated uses squared Euclidean distance and the arithmetic mean. Keep those two together.

Same points, two values of k.

k = 2

Centroids 1.5 and 10.5

Squared error sums to 1

A real partition

k = 4

Each point is a centroid

Squared error sums to 0

Perfect and useless

### Sum of squares for the two pairs

1. Errors from 1.5 are 0.5 and 0.5. Each square is 0.25. The pair contributes 0.5.

2. Errors from 10.5 are 0.5 and 0.5. That pair also contributes 0.5. The total is 1.

3. The mean of all four points is 6, and one cluster around 6 has sum of squares 82. You do not compare 1 with 82 across different k as if they were the same contest.

Problem 1. Centroids are 1.5 and 10.5 for points 1, 2, 10, and 11. What is the within-cluster sum of squares?

Answer. 1. Each of the four errors is 0.5, each square is 0.25, and 4 times 0.25 is 1.

Problem 2. Why is a sum of squares of 0 at k = 4 not a reason to choose k = 4, and why is a sum of 1 on raw units not comparable to a sum of 1 after you standardize?

Answer. At k = 4 every point is its own centroid, so the sum is 0 and the partition is useless. Standardization changes the units, so a sum of 1 before it and a sum of 1 after it are not the same quantity.

Lecture 6 · 90 minutes

## A split and its Gini

A decision tree splits rows with a question and repeats on each side. You pick the question that reduces impurity most. Gini impurity is 1 minus the sum of squared class fractions. On 6 rows with 4 yes and 2 no, Gini is 1 minus (4/6) squared minus (2/6) squared, which equals 4/9. A pure node has Gini 0. You stop on a pure node, at a depth cap, or when a node holds fewer than 2 rows.

Suppose a split makes a left node of 3 yes and 0 no, and a right node of 1 yes and 2 no. The left Gini is 0. The right Gini is 1 minus (1/3) squared minus (2/3) squared, which equals 4/9. The weighted impurity is (3/6) times 0 plus (3/6) times (4/9), which equals 2/9. The gain is the parent 4/9 minus 2/9, which is 2/9. You prefer this split to one whose gain is 0.

Parent impurity, then the two children.

Parent Gini is 4/9

Left child is pure, Gini 0

Right child Gini is 4/9

Gain after weighting is 2/9

### Gain on six labeled rows

1. The parent has 4 yes and 2 no. Gini is 4/9.

2. The left child has 3 yes. Its Gini is 0. The right child has 1 yes and 2 no. Its Gini is 4/9.

3. Weight each child by 3/6. Weighted impurity is 2/9. Gain is 4/9 minus 2/9, which is 2/9.

Problem 1. Six rows contain 4 yes and 2 no. What is the Gini impurity?

Answer. 4/9. Compute 1 minus (4/6) squared minus (2/6) squared, which is 1 minus 16/36 minus 4/36 = 16/36 = 4/9.

Problem 2. The split above has weighted impurity 2/9. What is the gain, and why is a gain computed on all 6 rows not yet a test score?

Answer. Gain is 4/9 minus 2/9 = 2/9. Those 6 rows are the rows you split on, so 2/9 is an in-sample reduction. It does not say what happens on rows you have not seen.

Lecture 7 · 90 minutes

## A holdout, not a training score

A score on the rows you trained on is not evidence. You hold out a test set before you look at it. With 50 rows, train on 40 and test on 10. A tree that memorizes training ids can score 40/40 on the train set and 5/10 on the test set. If the majority class covers 7 of the 10 test rows, a constant predictor scores 7/10 without using any feature. Your tree lost to that constant.

You do not tune the depth cap on the test set and then report that same test set as confirmation. Every time you reject a depth because of those 10 rows, the test set becomes part of training. Use a third slice, or accept that a single split of 50 rows is noisy. A rule you find by mining, such as a frequent itemset, needs the same discipline if you want to claim it will hold on new baskets.

Train score against a constant rule.

Memorizing tree

Train accuracy 40/40

Test accuracy 5/10

Uses the training ids

Majority class

Ignores every feature

Test accuracy 7/10

Wins on these 10 rows

### Forty rows to fit, ten to test

1. You randomly hold out 10 of 50 rows and fit only on the other 40.

2. The tree is perfect on the 40 and correct on 5 of the 10. The majority class is correct on 7 of the 10.

3. 5/10 is worse than 7/10. You do not ship the tree on the strength of the 40/40.

Problem 1. A tree scores 40/40 on the training rows and 5/10 on the test rows. The majority class scores 7/10 on that same test. Which predictor is better on the test, and why is 40/40 not the number you report?

Answer. The majority class is better on the test, 7/10 against 5/10. The 40/40 is a training score. A score on the rows you trained on is not evidence about new rows.

Problem 2. You try five depth caps and keep the one that looks best on the same 10 test rows. What did those 10 rows become?

Answer. They became part of training. Each rejection used them to choose the cap, so you cannot also report them as untouched confirmation. Use a third slice, or admit that one split of 50 rows is noisy.

Lecture 8 · 90 minutes

# A result you can recompute

You can now describe a table, mine frequent itemsets, score a rule by confidence and lift, cluster points with k-means, and grow a tree by Gini gain. You can also refuse a training score that was not checked on held-out rows. Itemset support answers a basket question. A tree answers a labeled question. A cluster id is not a class unless rows were labeled. Confidence on the same 4 baskets that produced the rule is not a test.

Two ideas have to stay attached when you write up a result. A pair that meets minimum count 2 in 4 baskets can still have lift 8/9, under 1, so frequency did not give you a dependence. A k-means sum of squares of 1 on standardized coordinates is not comparable to a sum of squares of 1 on raw coordinates, because the units changed. You name the threshold, the distance, and the split that produced the figure.

What you can now defend.

Support, then confidence and lift

A k you justify by error

A split scored by Gini gain

A number from held-out rows

## Frequency and a centroid in one writeup

1. The pair milk and bread meets count 2. Confidence is 2/3 and lift is 8/9. You report the lift, not only the count.

2. The line of points converges to centroids 1.5 and 10.5 with sum of squares 1, after you have said the coordinates are already on one scale.

3. If those points were still in mixed units, the 1 is not comparable to a 1 computed after standardization. The writeup names which sum it is.

Problem 1. Milk and bread occur together in 2 of 4 baskets, milk occurs in 3, and bread occurs in 3. The pair meets minimum count 2. What are confidence and lift of milk implies bread, and does the count alone show dependence?

Answer. Confidence is 2/3. Lift is (2/3) / (3/4) = 8/9, which is under 1. The count says the pair is frequent. The lift says it restates the base rate, so the count alone is not a dependence.

Problem 2. Your k-means sum of squares is 1 at centroids 1.5 and 10.5, and a tree's Gini gain on all 6 labeled rows is 2/9. Which of those numbers, if either, is a test-set accuracy?

Answer. Neither. The sum of squares is a training fit of the four points, and it is comparable only at a fixed k and a fixed scaling. The gain 2/9 is an in-sample impurity drop on the same 6 rows you split. A test accuracy needs rows you held out.